APPARELS VILLAGE LIMITED

System Architecture & RAG Integration Manual

A Technical Reference for the AVL Group Conversational Chatbot, Web Scraping Engine, and pgvector Indexing Pipeline

Document Reference:	AVL-CHAT-SYS-2026
Target System:	AVL Group Corporate Chatbot (https://avl.com.bd)
Technology Stack:	Django 5.x, DRF, PostgreSQL, pgvector, OpenAI API, Docker
Document Version:	v1.0.0
Date Published:	August 1, 2026
Authorship:	DeepMind Antigravity AI Pair Programming Agent

1. Executive Summary & Scope

Apparels Village Limited (AVL) is a premier 100% export-oriented apparel manufacturer based in Savar, Dhaka, Bangladesh. To streamline inquiries from global buyers, clients, and sourcing agents, AVL has deployed an interactive **AI Chatbot and Web Scraper System**. This system automatically crawls the official company website, indexes company facts (such as certifications, production capacities, machinery, and compliance audits), and supplies a conversational search engine powered by Retrieval-Augmented Generation (RAG).

The document details how the standard RAG pipeline stages—comprising user queries, document chunking, semantic vector generation, PostgreSQL indexing with pgvector, context-augmented prompt construction, and LLM text completion—are integrated into the Django and Docker application stack.

2. System Architecture & Data Flow

The AVL chatbot divides work into two discrete loops: the **Ingestion Loop** (crawling the site, chunking text, generating embeddings, and storing them) and the **Inference Loop** (receiving user questions, retrieving closest context chunks, querying the LLM, and streaming responses).

Ingestion Pipeline (Asynchronous / CLI)	Inference Pipeline (Real-Time API)
1. **Crawler Engine**: Crawls **`https://avl.com.bd`** recursively. 2. **HTML Cleanup**: Removes navigation, headers, footers, scripts via **`BeautifulSoup4`**. Parses image alt-tags for certificates. 3. **Semantic Chunking**: Breaks text into < 1200 char blocks based on paragraphs and headers. 4. **Vector Generation**: Converts chunks to 1536-dim vectors via OpenAI **`text-embedding-3-small`**. 5. **PostgreSQL Storage**: Saves vector models inside Postgres **`pgvector`** tables.	1. **User Query**: Receives question from dark-mode frontend SPA via **`/api/chat/`**. 2. **Query Vectorization**: Converts query to 1536-dim embedding via OpenAI. 3. **Similarity Retrieval**: Queries PostgreSQL using **`CosineDistance`** operator to pull the top $k=8$ closest chunks. 4. **Prompt Augmentation**: Combines System Persona guidelines, retrieved chunks, chat history (last 10 messages), and current query. 5. **LLM Generation**: Requests completion from **`gpt-4o-mini`** and returns JSON.

3. Detailed RAG Pipeline Step Mapping

Below is a matrix linking each theoretical step in the RAG Pipeline diagram directly to the file registry, data models, functions, and technologies implemented in the **avl-chat-bot** repository:

#	RAG Pipeline Stage	Project Code / Technical Implementation Details
1	User Query	Submitted as a JSON payload to **`/api/chat/`** containing **`message`** and **`session_id`**. Handled by **`ChatView.post()`** in **`chatbot/views.py`**.
2	Query Preprocessing	Validated on the backend by **`ChatRequestSerializer`** in **`chatbot/serializers.py`**. Cleans input, checks session variables, and binds the request.
3	Embedding Model	The user's query text is sent to the OpenAI embeddings API via **`get_embedding(query_text)`** in **`chatbot/scraper.py`**, which outputs a 1536-dimensional array using **`text-embedding-3-small`**.
4	Vector Database	PostgreSQL database equipped with the **`pgvector`** extension. The chunk vectors are stored in the **`embedding`** column of the **`chatbot_pagechunk`** table (mapped via **`PageChunk`** model in **`chatbot/models.py`**).
5	Retriever	Executes similarity search on the database. Uses **`pgvector.django.CosineDistance`** in **`retrieve_relevant_context()`** to rank chunks by cosine distance against the query vector.
6	Top-k Chunks	Extracts the top $k=8$ chunks. The query is evaluated with **`[:top_k]`** slicing, which compiles the best candidate chunks from the database.
7	Context Augmentation	Assembles the retrieved chunks into a unified string. Chunks are demarcated with **`Source URL`**, **`Page Title`**, and **`Content Section`** text markers, and joined together via **`\n---\n`** separators.
8	Prompt Template	Constructs the final text instructions for OpenAI. Merges the system instructions (defining role, compliance guidelines, tone constraints, and retrieved context chunks) with the last 10 messages of conversation history and the active user message.
9	Large Language Model	Invokes OpenAI's **`gpt-4o-mini`** model via **`client.chat.completions.create`** in **`chatbot/views.py`** with **`stream=False`** and a low temperature (**`0.2`**) to prevent hallucination.
10	Generated Response	The generated response text is appended to the user session's **`chat_details`** (JSONB) log for persistence, and returned in the HTTP Response payload.
11	Evaluation & Feedback	Session queries are logged. Telemetry dashboard endpoints **`/api/status/`** return database statistics, scraped pages list, last crawling timestamps, and scraping status flags.

4. Ingestion Pipeline: Crawling, Cleaning & Chunking

Data ingestion operates asynchronously via the Django management command `python manage.py scrape_website` or through the REST API route `/api/scrape/` which spins up a background daemon thread in Django. The logic in `chatbot/scraper.py` performs three major stages:

4.1 Recursive Domain Crawler & HTML Extraction

Starting at `https://avl.com.bd`, the scraper maps internal links belonging to the domain and follows them recursively. It sets a fake user agent to prevent request blockage, and skips non-HTML resources (CSS, JS, PDF, images, feeds). Once a page is fetched, `BeautifulSoup4` strips out navigational headers, dropdown menus, search bars, and footer blocks. This ensures that only structural elements—such as `<h1>` through `<h6>`, `<p>`, `<li>`, and tables—are scraped.

4.2 Metadata Enrichment: Image Alt & Certificate Parsing

Crucial compliance data in apparel industries is often embedded inside image files (logos, audits, certificates). The crawler has custom code to inspect elements having attributes like `data-elementor-lightbox-title` or standard `alt` / `title` tags on `<img>` elements. If it detects standard apparel certifications (such as **GOTS, OEKO-TEX, GRS, OCS, RCS, BSCI, SEDEX**), it expands the metadata: for example, mapping *'gots'* to *'Apparels Village Limited (AVL) holds the certification / membership: GOTS (Global Organic Textile Standard)'* and appends it directly to the text chunk. This guarantees compliance facts are searchable by the retrieval engine.

4.3 Semantic Chunking Logic

Standard RAG models require chunks of restricted length to avoid diluting LLM attention. The system applies a custom parser (`chunk_text()` in `scraper.py`) that executes semantic splits as follows:

- **Paragraph boundaries:** Splits text by double-line breaks (`\n`).
- **Heading Triggers:** If the running chunk size is greater than 150 characters, a new heading element (e.g. `H1:`, `H2:`, `# `) forces a chunk split, isolating the upcoming topic.
- **Length Cap:** The max size of any chunk is capped at 1200 characters. Paragraphs longer than this are split recursively at sentence endings (`[.!?]`).

4.4 Embedding Generation & File Export

Once scraping concludes, pages are written to the root file `scraped_content.md` as an audit log. Then, the import pipeline (`import_markdown_to_db()`) reads the file, executes chunking, requests 1536-dimensional float vectors from OpenAI's `text-embedding-3-small` endpoint, and commits them into PostgreSQL.

5. Database Schema & pgvector Integration

To query vectors natively inside relational SQL tables, the application integrates **`pgvector`** with Django's ORM.

Model Definitions (chatbot/models.py):

```
class PageChunk(models.Model):
    page = models.ForeignKey(ScrapedPage, related_name='chunks', on_delete=models.CASCADE)
    chunk_text = models.TextField()
    embedding = VectorField(dimensions=1536, null=True, blank=True)

class UserChatSession(models.Model):
    session_id = models.CharField(max_length=100, primary_key=True)
    user = models.ForeignKey(ChatUser, on_delete=models.CASCADE, db_column='user_id', null=True, blank
=True)
    chat_details = models.JSONField(default=list) # Persists user & assistant history
```

6. Inference Pipeline: Search, Prompting & Generation

When an API client submits a query to `/api/chat/`, the system runs a fast, real-time retrieval and generation loop:

6.1 Vector Similarity Search

The incoming user query is embedded into a 1536-dimensional float vector. The backend queries PostgreSQL to calculate the cosine distance against all page chunks, order by distance, and select the top 8 chunks. In Django, this is written as:

```
from pgvector.django import CosineDistance

query_vector = get_embedding(query_text)
top_chunks = PageChunk.objects.order_by(
    CosineDistance('embedding', query_vector)
)[:top_k]
```

6.2 Chat State, Session History & Inactivity Expiration

To allow conversational memory (follow-up questions), the backend maintains session history. The system extracts the last 10 messages of the conversation from `UserChatSession.chat_details` (stored in JSONB format) and appends them sequentially to the messages array sent to OpenAI.

Inactivity Timeout (10 Minutes): To maintain security and optimize server memory, sessions automatically expire if the user remains offline/inactive for 10 minutes. Both GET (session lookup) and POST (chat message submission) endpoints calculate the elapsed time since the session's last update timestamp. If more than 10 minutes have passed, the session is deleted from the PostgreSQL database, and a 404 response is returned. This prompts the client-side SPA to reset local session tokens and redirect the user back to registration.

6.3 System Prompt Design & LLM Call

The system prompt defines a highly specific persona: the professional, warm AI representative for AVL Group. It appends the compiled website context, and establishes strict rules:

1. **Strict Context Adherence:** Only answer questions using the provided context. If a detail is missing, say so, and suggest sending an email to `avl@faiyaz-group.com`.
2. **Human-like Tone:** Answer conversationally. Never use robotic phrases like 'According to the context...', 'Based on the database...', etc. State facts directly.
3. **Formatting Constraints:** Keep paragraphs short and concise. Under no circumstances output bullet points or numbered lists; instead, list items in a single cohesive, natural paragraph separated by commas.

7. Technology Stack Summary

The following diagram captures the physical technologies deployed in the Docker stack of the project:

Layer / Component	Technology / Package	Role in the Chatbot System
Web & API Framework	Django 5.x, DRF	Implements HTTP servers, REST APIs, JSON validation, and Django ORM database controls.

Database Server	PostgreSQL 16	Relational database storing scraped pages, chat sessions, user info, and logs.
Vector Search Module	pgvector Extension	Enables indexing of float arrays and SQL-based similarity searches in PostgreSQL.
LLM & Embeddings API	OpenAI API	Generates semantic vectors (text-embedding-3-small) and answers questions (gpt-4o-mini).
HTML Parsing Engine	BeautifulSoup4	Strips DOM elements, parses structures, and scrapes text content from the site.
Container Stack	Docker & Compose	Binds web server, Postgres DB, and pgAdmin containers into a unified service grid.
Interactive UI Client	HTML5, CSS3, JS	Single-Page dark mode user interface featuring suggestions and scrape controls.